

Day 1 - Strings

ITSRUNTYM

Strings Bootcamp – Complete Foundations (Java + C++)

1. What is an Array? (Java vs C++)

Java

```
int[] nums = {1, 2, 3, 4};  
char[] chars = {'a', 'b', 'c'};
```

- Arrays are **fixed-size** containers.
- Can store **primitive types** (int, char) or **objects** (e.g., String[]).
- Index starts from 0.

•

C++

```
int nums[] = {1, 2, 3, 4};  
char chars[] = {'a', 'b', 'c'};
```

- Arrays can be **stack-allocated** or **dynamically allocated**.
- Use `std::vector` for dynamic arrays.

•

2. What is a String?

Java

- Sequence of characters (immutable).

•

```
String s = "hello";
```

C++

- Uses **null-terminated character arrays** (`char[]`) or `std::string` (mutable).

•

```
char s[] = "hello";    // C-style string  
std::string str = "hello"; // C++ string
```

Difference

• Feature	• Java String	• C++ char[]	• C++ std::string
• Mutability	• Immutable	• Mutable	• Mutable
• Memory	• Heap (String Pool)	• Stack/Heap	• Heap
• Built-in methods	• Yes	• No	• Yes
• Null-termination	• No	• Yes (\0 ends it)	• No

3. String Creation (Java vs C++)

Java

```
String s1 = "hello";           // String literal
String s2 = new String("hello"); // New object (heap)
char[] arr = {'h','e','l','l','o'};
String s3 = new String(arr);
```

C++

```
char s1[] = "hello";           // C-style
std::string s2 = "hello";      // C++ string
```

Java String Pool Concept:

• Feature	• Stack Memory	• Heap Memory
• Purpose	• Stores method calls, local variables	• Stores objects, class instances, arrays
• Size	• Limited, small (per thread)	• Large, shared across threads

The **String Constant Pool** (also called the **intern pool**) is a **special area in the Java heap memory** where Java stores **string literals to save memory**.

```
String str1 = "Hello";
```

```
String str1 = "Hello";  
String str2 = "Hello";
```

```
String str1 = new String("John");  
String str2 = new String("Deo");
```

```
public static void main(String args[])  
{  
  
    // Declaring Strings using String literals  
    String s1 = "TAT";  
    String s2 = "TAT";  
  
    // Declaring Strings using new keyword  
    String s3 = new String("TAT");  
    String s4 = new String("TAT");  
  
    // Printing all the Strings  
    System.out.println(s1);  
    System.out.println(s2);  
    System.out.println(s3);  
    System.out.println(s4);  
}
```

```
String a = "hello";  
String b = "hello";  
System.out.println(a == b); // true (same pool object)  
String c = new String("hello");  
System.out.println(a == c); // false (different object)
```

Use .equals() for content comparison in Java.

Example in C++:

```
void function() {  
    int a = 5;           // Stack
```

```
int* ptr = new int(); // ptr in Stack, int() in Heap
}
```

a: allocated on the **stack**

- ptr: allocated on **stack**
- *ptr: allocated on **heap** (needs delete ptr to avoid memory leak)

•

4. String Immutability (Java)

- **Immutable (String)**: Every change creates a **new object**.
- **Mutable (StringBuilder/StringBuffer)**: Modifies the **same object**.

•

String vs StringBuilder vs StringBuffer in Java

• Feature	• String	• StringBuilder	• StringBuffer
• Mutability	• Immutable	• Mutable	• Mutable
• Thread-Safe	• No	• No	• Yes (synchronized)
• Performance	• Slowest (due to immutability)	• Fastest	• Slower than StringBuilder
• Use Case	• Read-only strings	• When thread-safety not needed	• Multi-threaded env
• Package	• java.lang	• java.lang	• java.lang

•

Examples and Behavior

1. String – Immutable

```
String s = "Hello";
s.concat(" World");
System.out.println(s); // Output: Hello
s = s.concat(" World");
System.out.println(s); // Output: Hello World
```

concat() created a new object but s wasn't updated until reassigned.

- Every operation = new object = more memory & time.

•

2. StringBuilder – Mutable and Not Thread-Safe

```
StringBuilder sb = new StringBuilder("Hello");  
sb.append(" World");  
System.out.println(sb); // Output: Hello World
```

- Modifies original object.
- Not thread-safe — don't use in concurrent programs without manual sync.

●

3. StringBuffer – Mutable and Thread-Safe

```
StringBuffer sbf = new StringBuffer("Hello");  
sbf.append(" World");  
System.out.println(sbf); // Output: Hello World
```

- Modifies original object.
- Thread-safe using synchronized methods.
- Slightly slower due to synchronization overhead.

●

Thread Safety Difference

StringBuffer

```
public synchronized StringBuffer append(String str) {  
    // thread-safe  
}
```

StringBuilder

```
public StringBuilder append(String str) {  
    // not thread-safe  
}
```

In Summary

• Feature	• String	• StringBuilder	• StringBuffer
• Immutable	• Yes	• No	• No
• Thread-safe	• No	• No	• Yes
• Performance	• Slow	• Fastest	• Medium
• Use-case	• Constant strings	• Fast string modifications	• Thread-safe mods
• Memory-efficient	• No (new object every time)	• Yes	• Yes

Are these available in C++?

No exact equivalents like Java's StringBuilder or StringBuffer, but...

In C++:

• Java	• C++ Equivalent
• String (immutable)	• std::string (mutable)
• StringBuilder	• std::ostringstream
• StringBuffer	• Manual sync needed using mutex or use thread-safe libs

Example: std::ostringstream in C++

```
#include <iostream>
#include <sstream>
int main() {
    std::ostringstream oss;
    oss << "Hello";
    oss << " World";
    std::string result = oss.str();
    std::cout << result << std::endl; // Output: Hello World
}
```

- Efficient for **many concatenations**.
- No thread safety unless wrapped manually.

•

Thread-Safe Strings in C++

C++ `std::string` is **not thread-safe**.

You can use:

- `std::mutex` to guard access.
- `std::atomic<std::string>` (C++20 and beyond).
- Or wrap it inside a thread-safe class.

•

Final Advice for Interview and Real-World Use

• Scenario	• Use In Java
• Read-only strings	• String
• Frequent string modifications	• StringBuilder
• Frequent string modifications + multithreading	• StringBuffer
•	

5. Common String Methods in Java

• Method	• Description	• Java Example
• <code>length()</code>	• Get length	• <code>s.length()</code>
• <code>charAt(i)</code>	• Character at index	• <code>s.charAt(2)</code>
• <code>substring(i, j)</code>	• Substring	• <code>s.substring(1, 4)</code>
• <code>equals()</code> / <code>equalsIgnoreCase()</code>	• Compare	• <code>s1.equals(s2)</code>
• <code>compareTo()</code>	• Lexicographic comparison	• <code>s1.compareTo(s2)</code>
• <code>indexOf()</code> , <code>lastIndexOf()</code>	• First/last occurrence	• <code>s.indexOf('a')</code>
• <code>toUpperCase()</code> , <code>toLowerCase()</code>	• Case change	• <code>s.toUpperCase()</code>
• <code>trim()</code>	• Remove surrounding spaces	• <code>s.trim()</code>
• <code>replace(a, b)</code>	• Replace characters	• <code>s.replace('a','b')</code>
• <code>split(" ")</code>	• Split string	• <code>s.split(" ")</code>
• Important Notes: Available in all three:		

- `.length()`

- `.charAt(i)`
- `.append()`, `.insert()`, `.delete()`, `.reverse()` → available only in `StringBuilder` and `StringBuffer`.

●

NOT available in `StringBuilder` or `StringBuffer`:

- `.equals()` works **only for object comparison** (not value).
- `.substring()`, `.trim()`, `.split()` — **only for `String`**

●

Workaround:

You can **convert** `StringBuilder` or `StringBuffer` to `String` to use all `String` methods:

```
StringBuilder sb = new StringBuilder(" Hello ");
String result = sb.toString().trim().toUpperCase();
```

C++ Equivalent (`std::string`)

```
std::string s = "hello";
s.length();      // Length
s[0];            // charAt
s.substr(1, 3);   // Substring
s == "hello";    // equals
s.find("l");      // indexOf
s.replace(1, 3, "XX"); // Replace
```

6. Concatenation & Performance

Java

```
String s = "hello" + " world"; // OK for few uses
String s = s + " again";       // Creates new object each time
```

```
// Efficient way
StringBuilder sb = new StringBuilder("hello");
sb.append(" world");
```

C++

```
std::string s = "hello";
```

```
s += " world"; // Efficient (mutable)
```

7. Iterating Over a String

Java

```
String s = "hello";  
for (int i = 0; i < s.length(); i++) {  
    System.out.print(s.charAt(i));  
}
```

C++

```
std::string s = "hello";  
for (char c : s) {  
    std::cout << c;  
}
```

8. Basic String Input/Output

Java

```
Scanner sc = new Scanner(System.in);  
String s = sc.nextLine(); // Reads full line including spaces
```

C++

```
std::string s;  
std::getline(std::cin, s); // Reads full line
```

9. Practice Problems (Beginner)

• Problem	• Java Approach Hint
• Reverse a string	• Loop from end to start or use StringBuilder
• Check if a string is a palindrome	• Two-pointer check
• Count vowels and consonants	• Use charAt(i) + check character
• Remove all spaces	• replace(" ", "")

• Convert string to char[]	• toCharArray()
• Frequency count	• Use HashMap<Character, Integer>
•	

Summary Table (Java vs C++)

• Feature	• Java	• C++
• String Class	• String, StringBuilder	• char[], std::string
• Mutability	• Immutable (String)	• Mutable (std::string)
• String Pool	• Yes	• No
• Memory Management	• GC (Garbage Collector)	• Manual or smart pointers
• Functions	• Built-in rich API	• Rich API in std::string
• Input with spaces	• Scanner.nextLine()	• getline(cin, str)

•